S-1 Sprint 1: Pathfinding Algorithm

Goal:

The goal is to create a 2d path-finding code on a simulated test map that will allow us to return from any point in the maze to a checkpoint or the beginning.

Research:

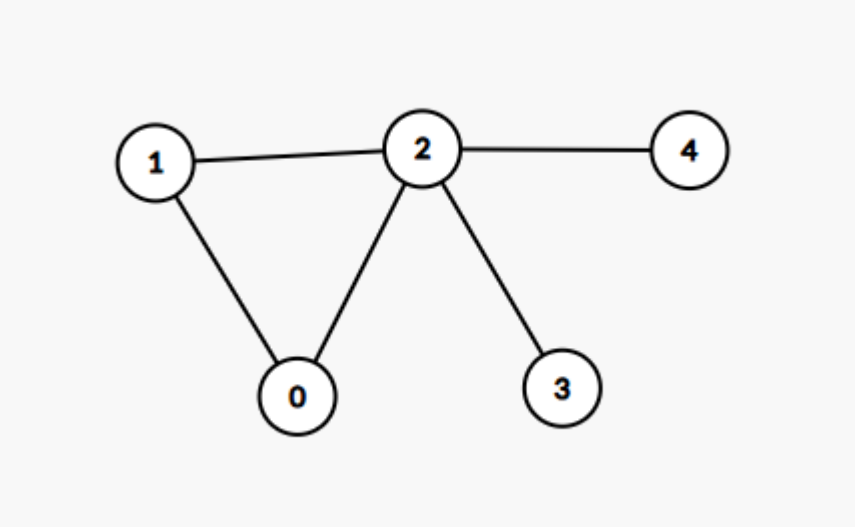
The most common pathfinding algorithms are DFS, BFS, and A*.

DFS

https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/

In DFS (or Depth First Search) for a graph, the algorithm traverses through all possible paths of through adjacent node before travelling to the next adjacent node. DFS will always use the first found path and contains a visited[] array to avoid revisiting nodes.

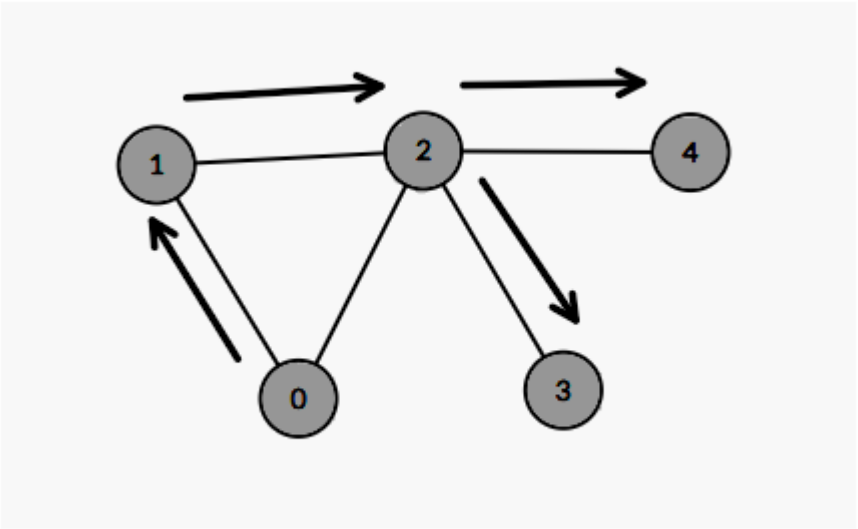
Example:



Node Map created using https://csacademy.com/app/graph_editor/

In this example, we start from node 0 and the goal is to reach node 4.

Data: [[0, 1], [0, 2], [1, 2], [2, 3], [2, 4]]



> For this example, we begin by traversing from node 0 to node 1, and from node 1 the only possible move is to node 2. At node 2, there are two adjacent nodes: 3 and 4. We move to node 3, which has no unvisited neighbors, so we backtrack to node 2. Finally, we proceed to node 4, which is our goal, at which point the traversal ends.

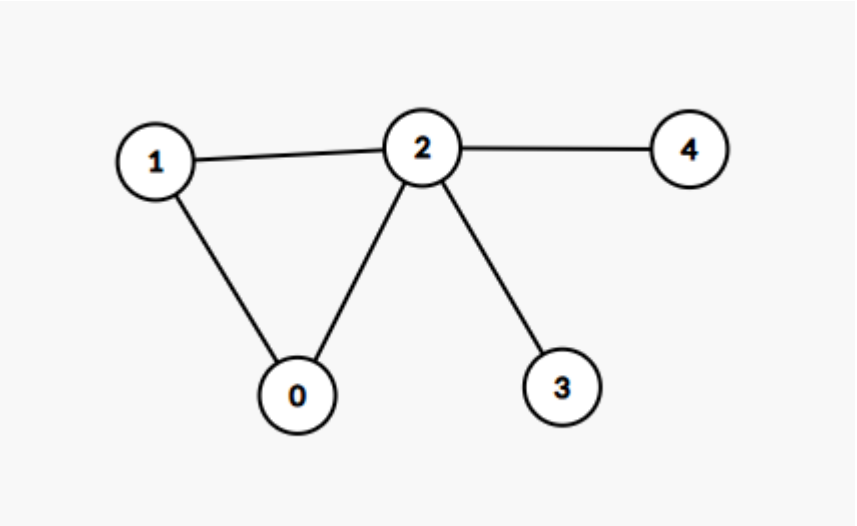
From this example, we can see that although DFS finds a valid path between two nodes, it does not necessarily find the most optimal one. Instead, it returns the first valid path it encounters. This does not achieve the goal we have set but is the easiest to implement.

BFS

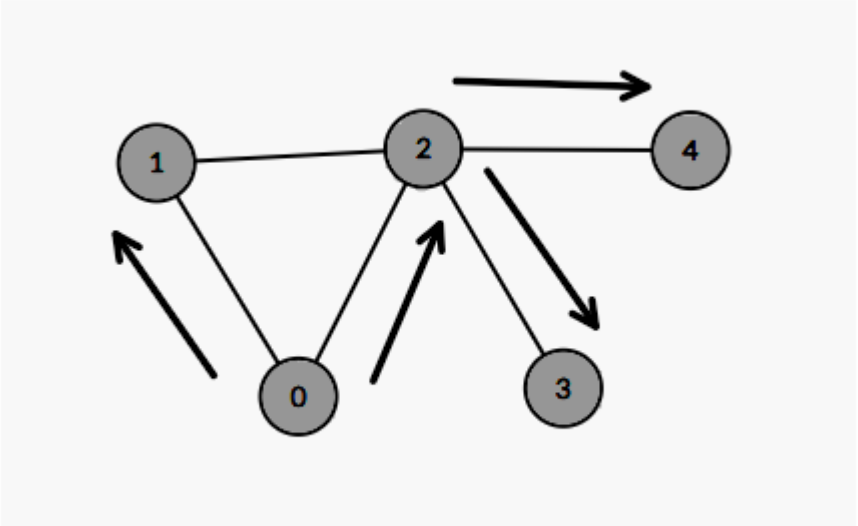
https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/

In BFS (Breadth First Search) for a graph, the algorithm traverses all nodes adjacent from the first node, then moves onto the next node. BFS traverses the whole graph, contains a visited[] array to avoid revisiting nodes, and contains a queue[] array to keep track of the order in which nodes should be processed.

Example:



Using the same example from the DFS, our goal is to traverse from node 0 to node 4.



Starting at node 0, nodes 1 and 2 are added to the queue, and node 0 is marked as visited. Next, node 1 is has its adjacent nodes either already visited or already in the queue, so no new nodes are added, and node 1 is marked as visited. Then, node 2 is processed, and it adds nodes 3 and 4 to the queue because nodes 0 and 1 have already been visited. Node 3 is then processed but has no unvisited neighbors. Finally, node 4 is reached, which is the target node, so the traversal ends.

A*

A* (A-star) search finds the most optimal path by considering both the distance traveled and an to the goal. It uses the formula f(n) = g(n) + h(n), where g(n) is the cost from the start node and h(n) is the estimated cost to the goal. At each step, A* selects the node with the lowest total cost f(n), ensuring that it prioritizes shorter paths. It keeps track of visited nodes and uses a priority queue to decide which node to explore next.

Our choice

Between the three algorithms, the choice came down to BFS and A. *We chose BFS because it was sufficient for the RoboCup maze challenge, while A* did not offer significant additional benefits beyond handling weighted costs for certain tiles (such as blue tiles, ramps, or obstacles).

Ideation:

I used a deque as the queue, along with 2D vectors to track visited nodes and parent nodes for path reconstruction. The algorithm starts by adding the starting position to the queue and marking it as visited. While the queue is not empty, it removes the front node and checks if it is the goal. If not, it adds all valid neighboring nodes to the queue. For debugging, I print each visited node and every node added to the queue. After the search, I print the visited array, and after reconstructing the path, I print the final path coordinates.

Prototype:

Version 1

```
#include <iostream>
#include <vector>
#include <deque>
using namespace std;

vector<pair<int,int>> BFS(pair<int,int> currentpos, vector<vector<bool>> map){
    //define queue, visited & path arrays
    deque<pair<int,int>> queue = {};
    size_t rows = map.size();
    size_t columns = map[0].size();
    vector<vector<bool>> visited(rows, vector<bool>(columns, false));
    vector<pair<int,int>> path = {};
    queue.push_back(currentpos);

    //search
    while(queue.size() > 0){ // While there is an element in queue.
        //mark as visited, print for debugging.
        visited[queue[0].first][queue[0].second] = true;
        cout << "visting: " << queue[0].first << "," << queue[0].second << endl;

        for(int i = 0; i < 4; i++){ //for each direction
            int nx = queue[0].first + dir[i][0];
            int ny = queue[0].second + dir[i][1];
            if(nx < map.size() && ny < map[0].size() && nx >= 0 && ny >= 0){ //check if in
bounds
                if(!visited[nx][ny] && !map[nx][ny]){ //check if the position is valid and
not visited
                    //
                    queue.push_back(pair<int,int>(nx,ny));
                    cout << "queue added: " << nx << "," << ny << endl;
                }
```

```
            }
        }
        queue.pop_front();
    }
    cout << "done search" << endl;
    //reconstruct (unfinished)
    return path;
}

int main()
{
    //2d map array, false tiles are traversable.
    vector<vector<bool> map = {
        {false,false,false},
        {false,true,false},
        {false,false,true},
        {true,false,false},
    }
}
```

This is the first version of the code. It uses a 2D boolean vector as the test map and does not include path reconstruction. The BFS implementation works correctly and visits all valid nodes.

Version 2

```
#include <iostream>
#include <vector>
#include <deque>
using namespace std;

const int MAP_SIZE = 20;

int dir[4][2] = { {1,0}, {-1,0}, {0,1}, {0,-1} };

deque<pair<int,int>> BFS(pair<int,int> currentpos, vector<vector<bool>> map, pair<int,int>
endpos) {
    //define queue, visited & path arrays
    deque<pair<int,int>> queue;

    size_t rows = map.size();
    size_t columns = map[0].size();

    vector<vector<bool>> visited(rows, vector<bool>(columns, false));
    deque<pair<int,int>> path;
    vector<vector<pair<int,int>>> prev(MAP_SIZE, vector<pair<int,int>>(MAP_SIZE));

    queue.push_back(currentpos);

    // search
    while (queue.size() > 0) { // While there is an element in queue.
        //mark as visited, print for debugging.
```

```
        visited[queue[0].first][queue[0].second] = true;

        int x = queue[0].first;
        int y = queue[0].second;

        cout << "visiting: " << x << ", " << y << endl;

        for (int i = 0; i < 4; i++) { //for each direction
            int nx = x + dir[i][0];
            int ny = y + dir[i][1];

            if (nx < map.size() && ny < map[0].size() && nx >= 0 && ny >= 0) { //check if in
bounds
                if (!visited[nx][ny] && map[nx][ny]) { //check if the position is valid and
not visited
                    queue.push_back(pair<int,int>(nx, ny));
                    // cout << "queue added: " << nx << ", " << ny << endl;
                    prev[nx][ny] = pair<int,int>(x, y);
                }
            }
        }
        //remove element from queue
        queue.pop_front();
    }

    // reconstruct -> essentially, each element contains coordinates that lead to the next
tile, allowing us to retrace our steps
    path.push_front(endpos);
    pair<int,int> curr = prev[endpos.first][endpos.second];

    while (true) {
        path.push_front(curr);
        curr = prev[curr.first][curr.second];

        if (path[0] == currentpos) {
            break;
        }
    }

    cout << endl << "done search" << endl << endl;

    return path;
}

int main() {
    //2d map array, false tiles are traversable.
    vector<vector<bool>> map = {
        {false, false, false},
        {false, true, false},
        {false, false, true},
        {true, false, false}
    };
```

```
    deque<pair<int,int>> path = BFS(pair<int,int>(0,0), map, pair<int,int>(3,2));

    //print path for debugging.
    cout << "Path: " << endl;
    for (int i = 0; i < path.size(); i++) {
        cout << path[i].first << ", " << path[i].second << endl;
    }
}
```

This version adds path reconstruction and returns the final path, and on the simple test map it consistently produces the fastest path. However, it does not implement the actual tilemap system defined by my teammates, so it is not fully integrated with the project's map structure. It also has a performance issue because visited nodes are not marked correctly at the right time, as they should be marked as visited as soon as they are added to the queue rather than when they are processed, which leads to repeated node insertions and significantly slows down the algorithm.

Final Design:

```
#include <iostream>
#include <vector>
#include <deque>
#include <utility>

using namespace std;

enum Direction {
    NORTH = 0,
    EAST = 1,
    SOUTH = 2,
    WEST = 3
};

enum TileTypes {
    BLANK = 0,
    BLUE = 1,
    BLACK = 2
};

template <unsigned size>
class Bitset
{
private:
    static constexpr unsigned W = 8; // bits per uint8_t
    static constexpr unsigned WORDS = (size + W - 1) / W;

    unsigned char mem[WORDS]{}; // zero-initialized

public:
    bool get(unsigned i) const
    {
```

```
        if (i >= size) return false;              // or throw/assert
        const unsigned word = i / W;
        const unsigned bit = i % W;
        return (mem[word] >> bit) & 0x1u;
    }

    void set(unsigned i, bool b)
    {
        if (i >= size) return;                    // or throw/assert
        const unsigned word = i / W;
        const unsigned bit = i % W;
        const unsigned char mask = static_cast<unsigned char>(1u << bit);

        if (b) mem[word] |= mask;
        else   mem[word] &= static_cast<unsigned char>(~mask);
    }
};

struct Tile {
    //bool discovered;
    //bool fullyExplored;


private:
    //first 4 bit is wall, last 4 bit is edge
    //set amount of bits
    Bitset<16> bitset;
    TileTypes tileType;
public:
    //get and set functions
    bool getWall(unsigned dir)
    {
        return bitset.get(dir);
    }
    void setWall(unsigned dir, bool stat) {
        bitset.set(dir, stat);
    }
    //+4 is index to the target bit
    bool getEdge(unsigned dir) {
        return bitset.get(dir + 4);
    }
    void setEdge(unsigned dir, bool stat) {
        bitset.set(dir + 4, stat);
    }
    bool getDiscovered() {
        return bitset.get(8);
    }

    void setDiscovered(bool stat) {
        bitset.set(8, stat);
    }
    bool getFully() {
        return bitset.get(9);
```

```
    }
    void setFully(bool stat) {
        bitset.set(9, stat);
    }
    TileTypes getType() {
        return tileType;
    }
    void setType(TileTypes type) {
        tileType = type;
    }
    bool getVictim() {
        return bitset.get(10);
    }
    void setVictim(bool vic) {
        bitset.set(10, vic);
    }

    //bool wall[4];
    //bool edge[4];


    //bool tileType
    //bool victim;

    //Tile();   // constructor
};

int dir[4][2] = {
    {0, 1},
    {1, 0},
    {0, -1},
    {-1, 0}
};

const int MAP_SIZE = 20;
Tile mapGrid[MAP_SIZE][MAP_SIZE];

void initializeMap(){
    for (int x = 0; x < MAP_SIZE; x++) {
        for (int y = 0; y < MAP_SIZE; y++) {
            mapGrid[x][y].setDiscovered(false);
            mapGrid[x][y].setFully(false);
            for (int d = 0; d < 4; d++) {
                if (x % 3 == 0 && y % 3 == 0) {
                    mapGrid[x][y].setWall(d, true);
                }
                mapGrid[x][y].setEdge(d, false);
            }
            mapGrid[x][y].setType(BLANK);
        }
    }
}
```

```
deque<pair<int,int>> BFS(pair<int,int> currentpos, Tile mapGrid[MAP_SIZE][MAP_SIZE],
pair<int, int> endpos) {
    //define queue, visited & path arrays
    deque<pair<int, int>> queue = {};
    size_t rows = MAP_SIZE;
    size_t columns = MAP_SIZE;
    vector<vector<bool>> visited(rows, vector<bool>(columns, false));
    deque<pair<int, int>> path = {};
    vector<vector<pair<int, int>>> prev(MAP_SIZE, vector<pair<int,int>>(MAP_SIZE));
    queue.push_back(currentpos);

    //search
    while (queue.size() > 0) { // While there is an element in queue.
        //mark as visited, print for debugging.
        int x = queue[0].first; int y = queue[0].second;
        //cout << "visting: " << x << "," << y << endl;

        for (int i = 0; i < 4; i++) { //for each direction
            int nx = x + dir[i][0];
            int ny = y + dir[i][1];
            if (nx < rows && ny < columns && nx >= 0 && ny >= 0) { //check if in bounds
                if (!visited[nx][ny] && !mapGrid[nx][ny].getWall(i)) { //IMPORTANT: ADD MORE
CONDITIONALS HERE
                    queue.push_back(pair<int, int>(nx, ny));
                    visited[nx][ny] = true;
                    //cout << "queue added: " << nx << "," << ny << endl;
                    prev[nx][ny] = pair<int, int>(x, y);
                    //cout << "at: " << nx << "," << ny << " added: " << x << "," << y <<
endl;
                }
            }
        }
        //remove element from queue
        queue.pop_front();
    }
    for (int i = 0; i < visited.size(); i++) {
        for (int j = 0; j < visited[0].size(); j++) {
            cout << visited[i][j] << " ";
        }
        cout << endl;
    }
    // reconstruct -> essentially, each element contains coordinates that lead to the next
tile, allowing us to retrace our steps
    path.push_front(endpos);
    pair<int, int> curr = prev[endpos.first][endpos.second];
    while (true) {
        //cout << curr.first << "," << curr.second << endl;
        path.push_front(curr);
        curr = prev[curr.first][curr.second];
        if (path[0] == currentpos) {
            break;
        }
```

```
    }
    //cout << endl << "done search" << endl << endl;
    return path;
}

int main()
{
    /*vector<vector<bool>> map = {
        {false, false, false},
        { false, true, false },
        { false, false ,true },
        { true, false ,false }
    };*/

    initializeMap();

    deque<pair<int, int>> path = BFS(pair<int, int>(0, 0), mapGrid, pair<int,int>(4,4));

    //print path for debugging.
    cout << "path: " << endl;
    for (int i = 0; i < path.size(); i++) {
        cout << path[i].first << "," << path[i].second << endl;
    }
}
```

The pathfinding system was built using a BFS (breadth-first search) approach on a 20×20 tilemap created with a Tile object array created by teammates. A direction array was defined to handle movement in the four cardinal directions during the search. The BFS function takes the current position, target position, and tilemap as inputs and returns a deque of integer pair positions representing the final path. Inside the function, a queue is used to manage exploration order, along with a 2D boolean visited array to track explored tiles and prevent revisiting, and a 2D vector called `prev` to store each node's parent for later path reconstruction. The algorithm begins by pushing the starting position into the queue, then repeatedly processes the front of the queue by checking its four neighboring tiles using the direction array. If a neighbor is within bounds, not visited, and not blocked by a wall, it is marked as visited, added to the queue, and its previous node is recorded in prev. This continues until the search completes, after which the path is reconstructed by tracing backwards from the target using the prev array, building a deque from end to start that represents the final shortest path.

```
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
path:
0,0
0,1
0,2
0,3
0,4
1,4
2,4
3,4
4,4
```

The code successfully performs pathfinding on a 20×20 test grid with no restrictions, and it completes the search quickly and correctly. In the map output, the tiles marked with 1 represent all the nodes that were visited during the search process, while the second output shows the final travelled path leading to the target position at (4,4). Overall, this indicates that the algorithm is working as intended.

```
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
path:
0,0
0,1
0,2
1,2
1,3
1,4
2,4
3,4
4,4
```

It also continues to work correctly when walled nodes are added to the test map, which shows that the pathfinding logic is handling obstacles properly and the algorithm successfully avoids the blocked areas while still finding a valid route to reach the target.

Conclusion

In conclusion, the system works correctly and reliably produces valid paths using BFS, even when obstacles are introduced into the tilemap. It successfully explores the grid, tracks visited nodes, and reconstructs the shortest path using the `prev` array without issues. To improve this in the future, the algorithm could be upgraded to A* pathfinding, which would make the search faster and more efficient as well as the implementation of weighted costs, such as higher movement penalties for blue tiles or elevation changes.